

第21回 Canvas の基本

実習時のフォルダ構成

第 20 回と同じ形式で、`Web_名前` の中に `21` フォルダを作る。`21` の中に `sample` と `exercise` を作り、次のファイルを用意する。

```
Web_名前
├── 21
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、すべて `sample/main.js` に書く）
- `exercise`：演習で作るファイル（演習 1～4、発展）

`sample/index.html` と `exercise/index.html` には、どちらも次の内容を書く。今回から、`<body>` の中に `<canvas>` を書く。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>Canvasの基本</title>
</head>
<body>
  <canvas id="game" width="400" height="500"></canvas>
  <script src="main.js"></script>
</body>
</html>
```

- `<canvas>`：JavaScript で絵を描くための「キャンバス（画用紙）」を、ページに置くタグ。
- `width="400" height="500"`：キャンバスの幅と高さ（単位はピクセル）。
- `id="game"`：JavaScript からこのキャンバスを見つけるための名前。

講義内容

以降の講義内容のコードは、`sample/main.js` に、上から順に書き足していく。書いたら `sample/index.html` をブラウザで開き、画面で結果を確認する。エラーは、今までどおり `F12` の開発者ツール（コンソール）に表示される。

1. 前回の復習

- 関数：処理をまとめて名前を付けたもの。 `function 名前(引数) { }` で作り、 `名前(値);` で呼び出す。 `return` で戻り値を返せる。
- オブジェクト：関係のある値をまとめたもの。 `{ x: 100, y: 400 }` のように作り、 `player.x` のように取り出す。

今回は、Canvas（キャンバス）を使って、画面に図形や文字を描く。これまではコンソールに文字を表示していたが、今回からは、ゲームの画面を描く。

2. Canvas を使う準備

```
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");
```

- `document.getElementById("game")`：HTML の中から、`id` が `game` の要素（ここでは `<canvas>`）を取り出す。`document` は、ブラウザに表示している HTML のページ全体のこと。
- `canvas.getContext("2d")`：キャンバスに絵を描くための道具（ペンや絵の具のようなもの）を取り出す。`ctx` は context（コンテキスト）の略。
- この授業では、この 2 行を「Canvas を使うときの決まった書き方」として、`main.js` の最初に書く。
- `getElementById` の `"game"` と、`<canvas>` の `id="game"` は、同じ名前にする。違うと、エラーになり何も描かれない。

3. 座標

キャンバスの位置は、左上を `(0, 0)` として、`x` 座標（横）と `y` 座標（縦）で表す。

(0, 0) —————→ x (右へ行くほど大きい)

■ (100, 50)

↓

y (下へ行くほど大きい) (400, 500) が右下

- `x` は右へ行くほど大きく、`y` は下へ行くほど大きい（数学のグラフとは上下が逆）。
- キャンバスの幅は `canvas.width`（400）、高さは `canvas.height`（500）で取り出せる。

4. 四角形を描く

```
ctx.fillStyle = "#000022";
ctx.fillRect(0, 0, canvas.width, canvas.height);
```

```
ctx.fillStyle = "#ff4444";
ctx.fillRect(50, 150, 80, 40);
```

- `ctx.fillStyle = "色";`：これから塗る色を決める。色は、CSS と同じ書き方（`#rrggbb` など）。

- `ctx.fillRect(x, y, 幅, 高さ);` : 左上が `(x, y)` の位置に、塗りつぶした四角形を描く。
- 1 つ目は、キャンバス全体 (`(0, 0)` から幅 400・高さ 500) を塗って、背景にしている。
- `ctx.fillRect()` のように、`ctx` に用意されている機能を使って描く (`console.log()` と同じ書き方)。

後から描いたものが、上に重なる。

```
ctx.fillStyle = "#ffff00";
ctx.fillRect(250, 280, 60, 60);
ctx.fillStyle = "#00ff00";
ctx.fillRect(280, 310, 60, 60);
```

- 黄色の四角形の上に、緑の四角形が重なって描かれる。描く順番に注意する（背景は最初に描く）。

5. 文字を描く

```
ctx.fillStyle = "#ffffff";
ctx.font = "20px sans-serif";
ctx.textAlign = "left";
ctx.fillText("SCORE: 0", 10, 30);

ctx.textAlign = "center";
ctx.fillText("SHOOTING GAME", canvas.width / 2, 250);
```

- `ctx.font = "大きさ フォント";` : 文字の大きさとフォントを決める。
- `ctx.fillText("文字", x, y);` : `(x, y)` の位置に文字を描く。`y` は、文字の**ベースライン**（ほぼ文字の下端。`g` や `y` などは、少し下にはみ出す）の位置。
- `ctx.textAlign` : `x` の位置に対して、文字をどちらにそろえるかを定める。

<code>textAlign</code>	<code>x</code> の位置が、文字の
"left"	左端になる
"center"	中央になる
"right"	右端になる

- `canvas.width / 2` は、キャンバスの横の真ん中 (200)。`"center"` と組み合わせると、文字を中央に置ける。
- 文字の中に変数の値を入れるときは、テンプレートリテラルが使える（第 22 回から使う）。

6. for 文で並べて描く

`for` 文の `i` を座標の計算に使うと、同じ形を並べて描ける。

```
ctx.fillStyle = "#ff4444";
for (let i = 0; i < 5; i++) {
  ctx.fillRect(40 + i * 70, 70, 32, 32);
}
```

	x 座標 <code>10 + i * 70</code>
0	40
1	110
2	180
3	250
4	320

7. オブジェクトと関数で描く

自機を、第 20 回で学んだオブジェクトで表し、描く処理を関数にまとめる。

```
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

drawPlayer();
```

- オブジェクトは、このように 1 行で書いてもよい。
- `player.x` や `player.y` の値を変えてから `drawPlayer()` を呼び出すと、違う位置に描ける。
- 次回から、この `player` と `drawPlayer` を使って、自機を動かす。`speed`（速さ）も、次回から使う。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて確認する。

演習 1 背景と敵を描く

- Canvas を使う準備の 2 行を書く。
- キャンバス全体を、好きな色で塗って背景にする。
- 上の方に、敵（32 × 32 の四角形）を 3 つ描く。色は自分で決める。

演習 2 for 文で並べて描く

- `for` 文を使って、小さな四角形を横に 6 つ並べて描く。
- もう 1 つ `for` 文を書いて、その下の段に、別の色で 6 つ並べて描く。

演習 3 文字を描く

- 左上に「SCORE: 0」と描く。
- 右上に「LIVES: 3」と描く（`textAlign` を `"right"` にし、`x` を `canvas.width - 10` にする）。

演習4 オブジェクトと関数で自機を描く

- 自機をオブジェクトで作る (`x`・`y`・`w`・`h`・`speed`。値は自分で決める)。
- 自機を描く関数 `drawPlayer` を作り、呼び出して、キャンバスの下の方に描く。
- `player.x` の値を変えて、描かれる位置が変わることを確認する。

発展（任意）

- 自機の周りに四角形を描き足して、翼や先端のある形にする。位置は `player.x`・`player.y` をもとに計算する（例：`player.x - 12`）。`player.x` を変えても、形がくずれずに一緒に動くことを確認する。
- `for` 文と乱数を使って、小さな白い四角形 (2×2) を 30 個、ランダムな位置に描いて、星空にする。再読み込みするたびに、星の位置が変わることを確認する。